

14.1 Introduction

People like you and I reside in our countries and live here. We can think of each country as a set and all of the people within it as elements within that set. The same person cannot live in two different countries simultaneously. What we have just modeled is a **disjoint set**.

Two sets are named *disjoint sets* if they have no elements in common. A Disjoint-Sets (or Union-Find) data structure keeps track of a fixed number of elements partitioned into a number of *disjoint sets*. The data structure has two operations:

1. `connect(x, y)`: connect x and y . Also known as union
2. `isConnected(x, y)`: returns true if x and y are connected (i.e. part of the same set).

A Disjoint Sets data structure has a fixed number of elements that each start out in their own subset. By calling `connect(x, y)` for some elements x and y , we merge subsets together.

For example, say we have four elements which we'll call A, B, C, D. To start off, each element is in its own set:



$\{A\} \{B\} \{C\} \{D\}$

After calling `connect(A, B)`:



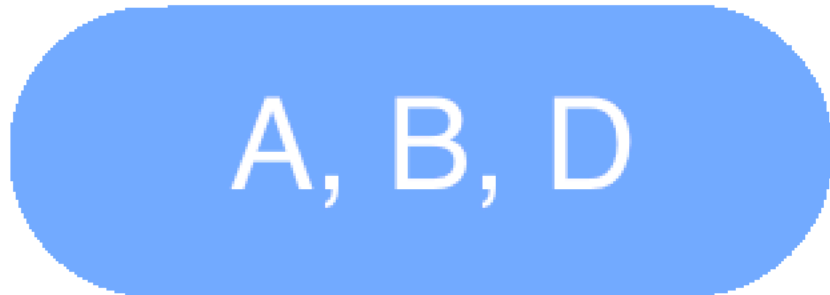
$\{A, B\} \{C\} \{D\}$

Note that the subsets A and B were merged. Let's check the output some `isConnected` calls:

`isConnected(A, B) -> true`

`isConnected(A, C) -> false`

After calling `connect(A, D)`:



$\{A, B, D\} \{C\}$

We find the set A is part of and merge it with the set D is part of, creating one big A, B, D set. C is left alone.

`isConnected(A, D) -> true`

`isConnected(A, C) -> false`

With this intuition in mind, let's formally define what our DisjointSets interface looks like. As a reminder, an **interface** determines *what* behaviors a data structure should have (but not *how* to accomplish it). In this way, any class that implements the DisjointSets interface knows to always include functions: `connect(int p, int q)` and `isConnected(int p, int q)` as seen below. For now, we'll only deal with sets of non-negative integers. This is not a limitation because in production we can assign integer values to anything we would like to represent.

```
public interface DisjointSets {
    /** connects two items P and Q */
    void connect(int p, int q);

    /** checks to see if two items are connected */
    boolean isConnected(int p, int q);
}
```

But how are we going to save data for these Disjoint sets to see which member belongs to it's corresponding set? What data structures are we going to use to represent this awesome data structure? In addition to learning about how to implement a fascinating data structure, this chapter will be a chance to see how an implementation of a data structure evolves. We will discuss four iterations of a Disjoint Sets design before being satisfied: [Quick Find](#) → [Quick Union](#) → [Weighted Quick Union \(WQU\)](#) → [WQU with Path Compression](#). We will see how design decisions greatly affect asymptotic runtime and code complexity.

14.2 Quick Find

List of Sets

Intuitively, we might first consider representing Disjoint Sets as a list of sets, e.g. `List<Set<Integer>>`.

For instance, if we have $N=6$ elements and nothing has been connected yet, our list of sets looks like: $\{\{0\}, \{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}\}$. Looks good. However, consider how to complete an operation like `connect(5, 6)`. We'd have to iterate through up to N sets to find 5 and N sets to find 6. Our runtime becomes $O(N)$. And, if you were to try and implement this, the code would be quite complex.

The lesson to take away is that **initial design decisions determine our code complexity and runtime**.

Quick Find

Let's consider another approach using a *single array of integers*.

- The **indices of the array** represent the elements of our set.
- The **value at an index** is the set number it belongs to.

For example, we represent $\{0, 1, 2, 4\}$, $\{3, 5\}$, $\{6\}$ as:

int[] id	4	4	4	5
	0	1	2	3

Set 4: $\{0, 1, 2, 4\}$ | Set 5: $\{3, 5\}$ | Set 6: $\{6\}$

The array indices (0...6) are the elements. The value at `id[i]` is the set it belongs to. *The specific set number doesn't matter as long as all elements in the same set share the same id.*

connect(x, y)

Let's see how the connect operation would work. Right now, `id[2] = 4` and `id[3] = 5`. After calling `connect(2, 3)`, all the elements with id 4 and 5 should have the same id. Let's assign them all the value 5 for now:

$\{0, 1, 2, 4,$

int[] id	5	5	5	5
	0	1	2	3

Set 5: {0, 1, 2, 3, 4, 5} | Set 6: {6}

`isConnected(x, y)`

To check `isConnected(x, y)`, we simply check if `id[x] == id[y]`. Note this is a constant time operation!

We call this implementation “Quick Find” because finding if elements are connected takes constant time.

Code & Runtimes

```
public class QuickFindDS implements DisjointSets {
    private int[] id;

    /*  $\Theta(N)$  */
    public QuickFindDS(int N){
        id = new int[N];
        for (int i = 0; i < N; i++){
            id[i] = i;
        }
    }

    /* need to iterate through the array =>  $\Theta(N)$  */
    public void connect(int p, int q){
        int pid = id[p];
        int qid = id[q];
        for (int i = 0; i < id.length; i++){
            if (id[i] == pid){
                id[i] = qid;
            }
        }
    }

    /*  $\Theta(1)$  */
    public boolean isConnected(int p, int q){
        return (id[p] == id[q]);
    }
}
```

N = number of elements in our DisjointSets data structure

Implementation	Constructor	connect	isConnected
ListOfSets	$\Theta(N)$	$O(N)$	$O(N)$
QuickFind	$\Theta(N)$	$\Theta(N)$	$\Theta(1)$

Note

1. We didn't discuss this but you can reason that having to create N distinct sets initially is $\Theta(N)$

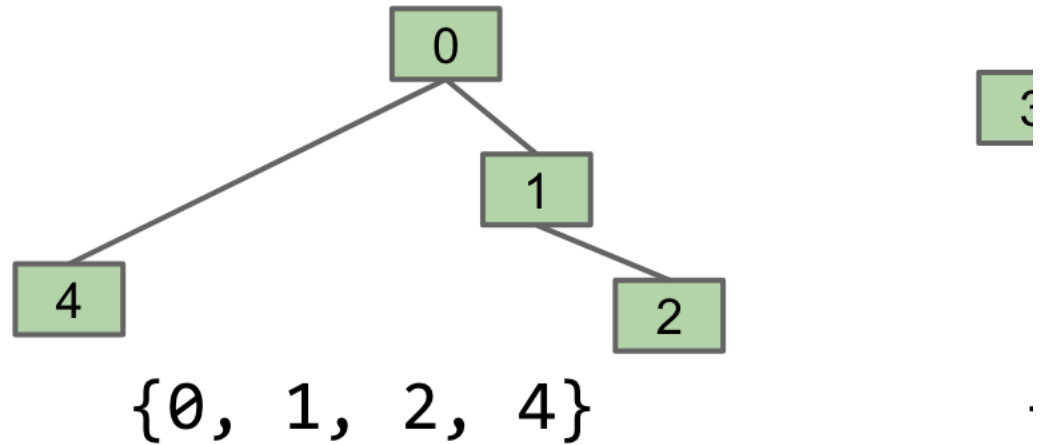
14.3 Quick Union

Suppose we prioritize making the connect operation fast. We will still represent our sets with an array. Instead of an id, we assign each item the index of its parent. If an item has no parent, then it is a 'root' and we assign it a negative value.

This approach allows us to imagine each of our sets as a tree. For example, we represent {0, 1, 2, 4}, {3, 5}, {6} as:

Note that we represent the sets using **only an array**. We visualize it ourselves as trees.

parent	-1	0	1	-1	0
	0	1	2	3	4



0 is the **root**
of this set.

3 is
of th

For QuickUnion we define a helper function `find(int item)` which returns the root of the tree item is in. For example, for the sets above, `find(4) == 0`, `find(1) == 0`, `find(5) == 3`, etc. Each element has a unique root.

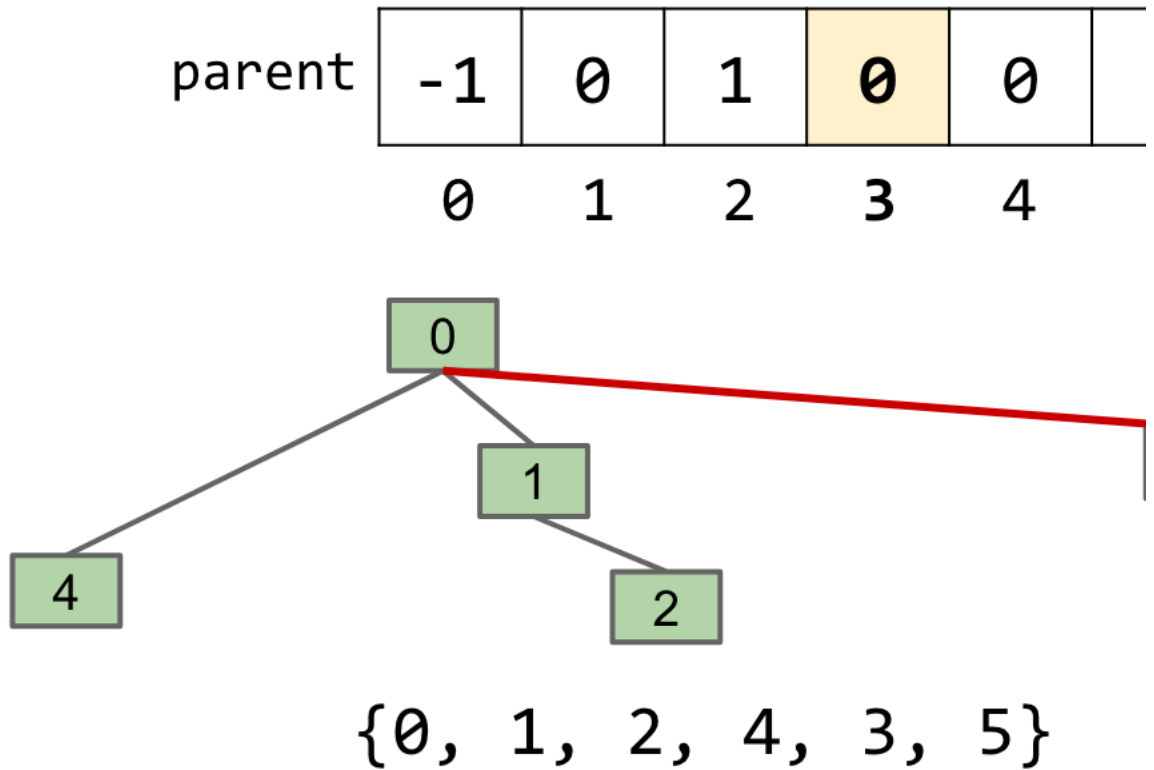
`connect(x, y)`

To connect two items, we find the set that each item belongs to (the roots of their respective trees), and make one the child of the other. Example:

`connect(5, 2):`

1. `find(5) -> 3`
2. `find(2) -> 0`
3. Set `find(5)`'s value to `find(2)` aka `parent[3] = 0`

Note how element 3 now points to element 0, combining the two trees/sets into one.



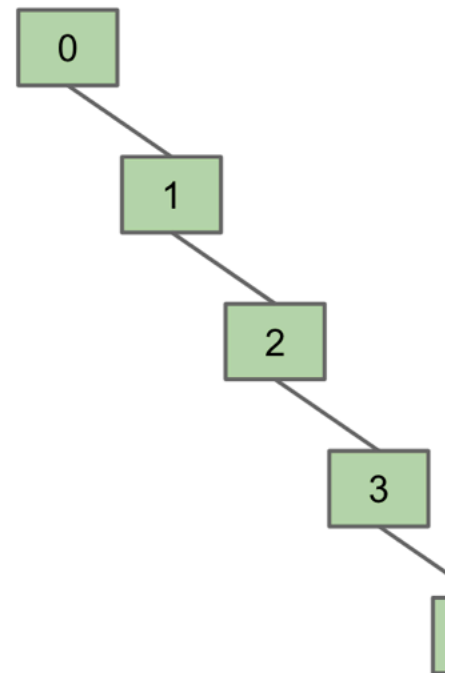
In the best case, if x and y are both roots of their trees, then `connect( $x$ ,  $y$ )` just makes x point to y , a $\Theta(1)$ operation! (Hence the name QuickUnion)

`isConnected( $x$ ,  $y$ )`

If two elements are part of the same set, then they will be in the same tree. Thus, they will have the same root. So for `isConnected( $x$ ,  $y$ )` we simply check if `find( $x$ ) == find( $y$ )`.

Performance

There is a potential performance issue with QuickUnion: the tree can become very long. In this case, finding the root of an item (`find(item)`) becomes very expensive. Consider the tree below:



In the worst case, we have to traverse all the items to get to the root, which is a $\Theta(N)$ runtime. Since we have to call `find` for both `connect` and `isConnected`, the runtime for both is upper bounded by $O(N)$.

Summary and Code

N = number of elements in our DisjointSets data structure

Implementation Constructor connect isConnected

QuickUnion	$\Theta(N)$	$O(N)$	$O(N)$
QuickFind	$\Theta(N)$	$\Theta(N)$	$\Theta(1)$
QuickUnion	$\Theta(N)$	$O(N)$	$O(N)$

From the runtime chart, QuickUnion seems worse than QuickFind! Note however that $O(N)$ as an **upper bound**. When our trees are balanced, both `connect` and `isConnected` perform reasonably well. In the next section we'll see how to *guarantee* they perform well.

```
public class QuickUnionDS implements DisjointSets {
    private int[] parent;

    public QuickUnionDS(int num) {
        parent = new int[num];
        for (int i = 0; i < num; i++) {
            parent[i] = -1;
        }
    }

    private int find(int p) {
        while (parent[p] >= 0) {
            p = parent[p];
        }
        return p;
    }

    @Override
    public void connect(int p, int q) {
        int i = find(p);
        int j = find(q);
        parent[i] = j;
    }

    @Override
    public boolean isConnected(int p, int q) {
        return find(p) == find(q);
    }
}
```

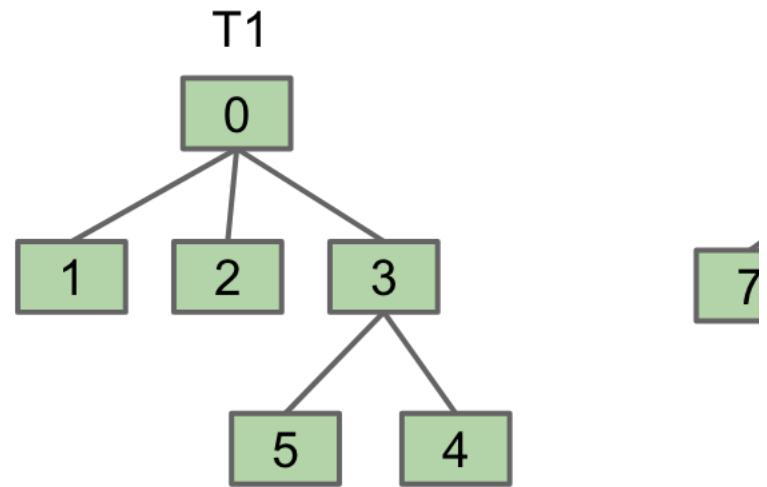
14.4 Weighted Quick Union (WQU)

Improving on Quick Union relies on a key insight: whenever we call `find`, we have to climb to the root of a tree. Thus, the shorter the tree the faster it takes!

New rule: whenever we call `connect`, we always link the root of the smaller tree to the larger tree.

Following this rule will give your trees a maximum height of $\log N$, where N is the number of elements in our Disjoint Sets. How does this affect the runtime of `connect` and `isConnected`?

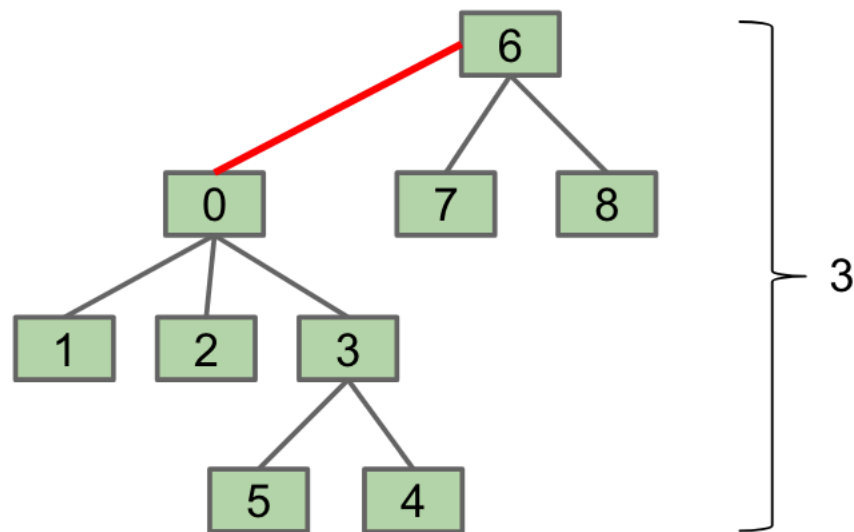
Let's illustrate the benefit of this with an example. Consider connecting the two sets T1 and T2 below:



Size: 6

We have two options for connecting them:

The first option we link T1 to T2. In the second, we link T2 to T1.



Option 1

The **second option is preferable** as it only has a height of 2, rather than 3. By our new rule, we would choose the second option as well because T2 is smaller than T1 (size of 3 compared to 6).

We determine smaller / larger by the number of items in a tree. Thus, when connecting two trees we need to know their size (or weight). We can store this information in the root of the tree by replacing the -1's with -(size of tree).

Maximum height: Log N

Following the above rule ensures that the *maximum* height of any tree is $\Theta(\log N)$. N is the number of elements in our Disjoint Sets. **By extension, the runtimes of connect and isConnected are bounded by $O(\log N)$.**

Why $\log N$? The video above presents a more visual explanation. Here's an optional mathematical explanation why the maximum height is $\log_2 N$. Imagine any element x in tree $T1$. The depth of x increases by 1 only when $T1$ is placed below another tree $T2$. When that happens, the size of the resulting tree will be at least double the size of

$T1$ because $size(T2) \geq size(T1)$. The tree with x can double at most $\log_2 N$ times until we've reached a total of N items ($2^{\log_2 N} = N$). So we can double up to $\log_2 N$ times and each time, our tree adds a level $\rightarrow$ maximum $\log_2 N$ levels.

You may be wondering why we don't link trees based off of height instead of weight. It turns out this is more complicated to implement and gives us the same $\Theta(\log N)$ height limit.

Summary

Implementation	Constructor	connect	isConnected
QuickUnion	$\Theta(N)$	$O(N)$	$O(N)$
QuickFind	$\Theta(N)$	$\Theta(N)$	$\Theta(1)$
QuickUnion	$\Theta(N)$	$O(N)$	$O(N)$
Weighted Quick Union	$\Theta(N)$	$O(\log N)$	$O(\log N)$

N = number of elements in our DisjointSets data structure

14.5 Weighted Quick Union with Path Compression

The clever insight is realizing that whenever we call `find(x)` we have to traverse the path from x to root. So, along the way we can connect all the items we visit to their root at no extra asymptotic cost.

Connecting all the items along the way to the root will help make our tree shorter with each call to `find`.

Recall that **both** `connect(x, y)` **and** `isConnected(x, y)` **always call** `find(x)` **and** `find(y)`. Thus, after calling `connect` or `isConnected` enough, essentially all elements will point directly to their root.

By extension, the average runtime of `connect` and `isConnected` becomes **almost constant** in the long term! This is called the *amortized runtime*.

More specifically, for M operations on N elements, WQU with Path Compression is in $O(N + M(\lg^* N))$. $\lg^*$ is the [iterated logarithm](#) which is less than 5 for any real-world input.

Summary

N : number of elements in Disjoint Set

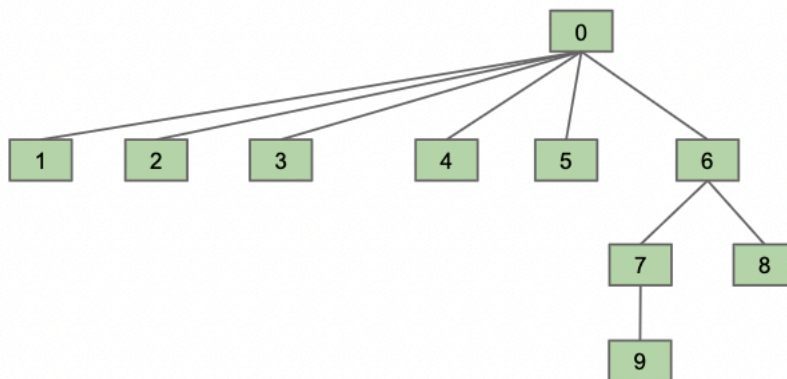
Implementation	isConnected	connect
Quick Find	$\Theta(1)$	$\Theta(N)$
Quick Union	$O(N)$	$O(N)$
Weighted Quick Union (WQU)	$O(\log N)$	$O(\log N)$
WQU with Path Compression	$O(\alpha(N))^*$	$O(\alpha(N))^*$

*behaves as constant in long term.

14.6 Exercises

Factual

1. [Problem 2](#) from the Spring 2016 Midterm 2.
2. [Problem 1d](#) from the Spring 2015 Midterm 2.
3. Suppose we have the following WQU with path compression. What is the height of the tree after we call `isConnected(8, 9)`?



- Problem 1
- ▼ Problem 2

[Solutions](#) are linked here and on the course website.

▼ Problem 3

The resulting tree will have height 1. Every node along the path from 0 to 9 will now have parent 0, and similarly every node along the path from 0 to 8 will also have parent 0.

Conceptual

- Which of the following arrays could represent a valid weighted quick union structure?
 - ☐ [8, 0, 4, 0, 0, 4, 0, 4, 2, 0]
 - ☐ [4, -8, 8, 2, 1, -2, 1, 1, 4, 5]
 - ☐ [3, 3, 5, 9, 3, 6, 3, 4, 1, -10]
 - ☐ [2, -10, 1, 1, 1, 1, 1, 2, 1, 7]

► Problem 1

Procedural

- Define a *fully connected* WQU as one where all elements are in the same set. What is the maximum and minimum height of a fully connected WQU with 6 elements?
- Suppose we have a WQU of height H . What is the minimum number of elements that must be in the WQU?

▼ Problem 1

The minimum height is always 1 (all elements are connected to the root). The maximum height is 2 (we take the floor of $\log_2 6$).

▼ Problem 2

We know that $H \leq \log_2 N$, where N is the number of elements in the WQU. As such, $N \geq 2^H$.

Metacognitive

- [Problem 3](#) from the Spring 2017 Midterm 2.
- Suppose we create a WQU with N items, then we perform M_C union operations and M_U union operations. Using big O notation, what is the runtime of this sequence of operations?
- Using the same variables as problem 2, describe a sequence of operations that would result in a runtime of $O(N + M_U + M_C)$.
- Write a `int find(int p)` method for the WQU with path compression. It should perform path compression as described in lecture: any node on the path from root to our target node should have its parent reset to the root. It takes in the target node p and returns the root of the tree p is in.

▼ Problem 1

[Solutions](#) are linked here and on the course website.

▼ Problem 2

$O(N + (M_U + M_C) \log N)$. Each operation takes $\log N$ time, and we need N time to initialize an empty array of size N .

▼ Problem 3

Initialize the array as usual. Then, connect the items in sequence: `connect(0, 1)`, `connect(0, 2)`, `connect(0, 3)`, up to `connect(0, N)`. This results in a height-1 tree, on which `connect` and `isConnected` run in constant time.

▼ Problem 4

```
public int find(int p) {
    int root = p;
    while (root != parent[root]) {
        root = parent[root];
    }

    while (p != root) {
        int newp = parent[p];
        parent[p] = root;
        p = newp;
    }

    return root;
}
```

14. Disjoint Sets